UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

cataLOG

Name: Todi Tinu-Constantin
Group: 30237

Table of Contents

Deliverable 1 ..... ***3***

Project Specification **3**

Functional Requirements **3**

Use Case Model **3**

Use Cases Identification: 3

UML Use Case Diagrams 4

Supplementary Specification **4**

Non-functional Requirements 4

Design Constraints 4

Glossary **5**

Deliverable 2 ..... ***5***

Domain Model **5**

Architectural Design **6**

Conceptual Architecture 6

Package Design 6

Component and Deployment Diagram 7

Deliverable 3 ..... ***9***

Design Model **9**

Dynamic Behavior 9

Class Diagram 9

Data Model **9**

System Testing ***9***

Future Improvements ***9***

Conclusion ***10***

Bibliography ***10***

Deliverable 1

Project Specification

This project aims to create an application that proposes the management of a student catalog. Within this application, various functionalities are available, depending on the role of the user. The frontend is written in : HTML and JavaScript, using Razor Pages, and the backend is written in C#, using ASP.NET and Entity Framework. Data persistence is ensured by the database connection, managed with Microsoft SQL Server.

.Functional Requirements

Depending on the role of the user, several functionalities are available. There are 4 user-roles: Student, Teacher, Anonymus User and Admin. They have the same basic functionalities (except Anonymus User), like Login, Logout, Register, etc.

1) Student
   - Login
   - Logout
   - Delete Account
   - Edit Profile
   - See the grades
   - Register
   - See Account Details
   - Search and See other registered users

2) Teacher
   - Login
   - Logout
   - Delete Account
   - Edit Profile
   - Grade the student
   - Register
   - See Account Details
   - Search and See other registered users

3) Anonymus User
   - Can search and see registered users, but some user details are hidden

4) Admin

   - Delete registered users if they misconduct

Use Case Model

Use Cases Identification:

Use-Case : Teacher grades a Student

Level : Teacher can grade a student

Primary Actor : Teacher

Main success scenario : Teacher grades the student and the Student can see the grades

Extension : In case of error, the student cannot see the grade.

Use-Case: User registration

Level : Anonymus User can sign up

Primary Actor : Unregistered User

Main success scenario : The database will have a new user

Extension : In case of error, the unregistered user is not registered and he is not in the database.

Use-Case : User Login

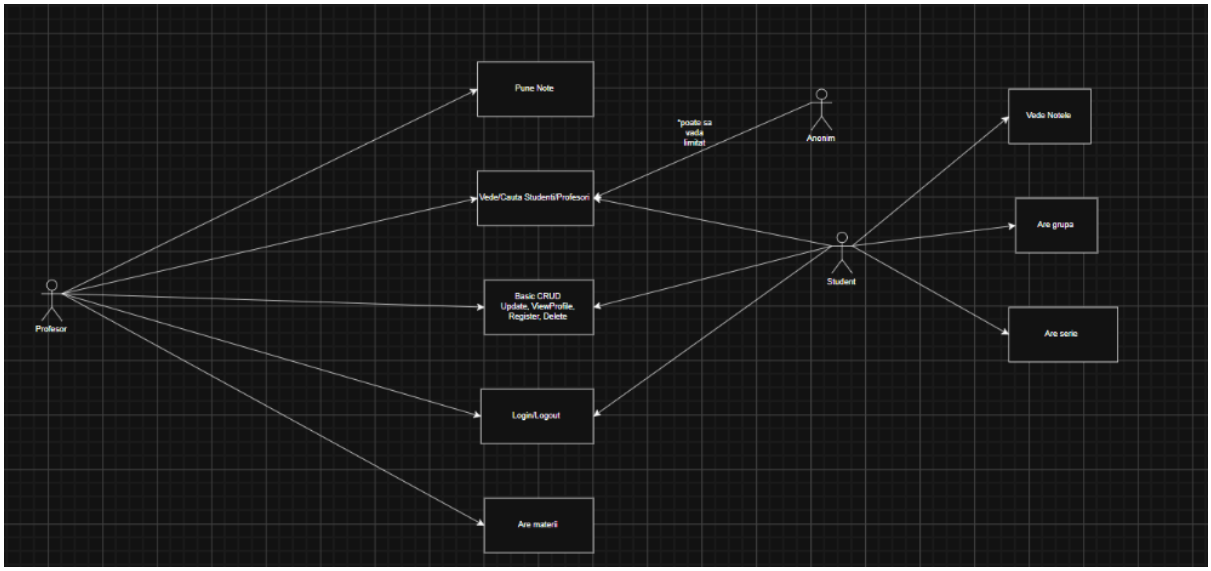
Level : User wants to go into the application and he is ALREADY registered.

Primary Actor: Registered User(Student or Teacher)

Main success scenario : The user can go into the role designed dashboard.

Extension : In case of error, the user cannot access the main dashboard.

UML Use Case Diagrams



Supplementary Specification

Non-functional Requirements

1) Security → Multiple Security checks, every account decision (like delete, edit details, etc) requires the password.
2) Capacity → Microsoft SQL Server maximum capacity
3) Technology → The system uses Microsoft SQL Server.
4) Reability and Availability → The system shall not have errors.
5) Compatibility → It is compatible with all systems and can be implemented everywhere.

Design Constraints

The system is built in Visual Studio IDE. The backend part is built in C#, using ASP.NET and Entity. Exista 4 nivele in cadrul arhitecturii : Model, ViewModel, View si Controller. The frontend part is built using HTML and JavaScript, using Razor Pages. Data persistence is ensured by the connection to a database, specifically to Micosoft SQL Server.

Glossary

Framework → supporting structure on which other things are built.

Backend → in software development, backend refers to the data management and processing behind the scenes.

Frontend → in software development, frontend refers to the presentation layer that users interact with.

Data persistence → data persistence means that data is saved permanently, so it is not lost when the system shuts down or restarts.

ASP.NET → is a server-side web application framework designed for web development to produce dynamic web pages. It was developed by Microsoft. ASP stands for Active Server Pages Network Enabled Technologies.

Entity → is a open source object-relational mapping framework for ADO.NET (.NET family)

C# → is a general-purpose high-level programming language supporting multiple paradigms. It encompasses static typing, strong typing, imperative, declarative, functional, generic, OOP. Developed by Microsoft.

Razor Pages → Razor is an ASP.NET programming syntax used to create to dynamic web pages with C# or VB.NET programming languages. Razor is a simple-syntax view engine and was released as part of MVC3 and the WebMatrix tool set.

HTML → is the standard markup language for documents designed to be displayed in a web browser. It defines the content and structure of web content.

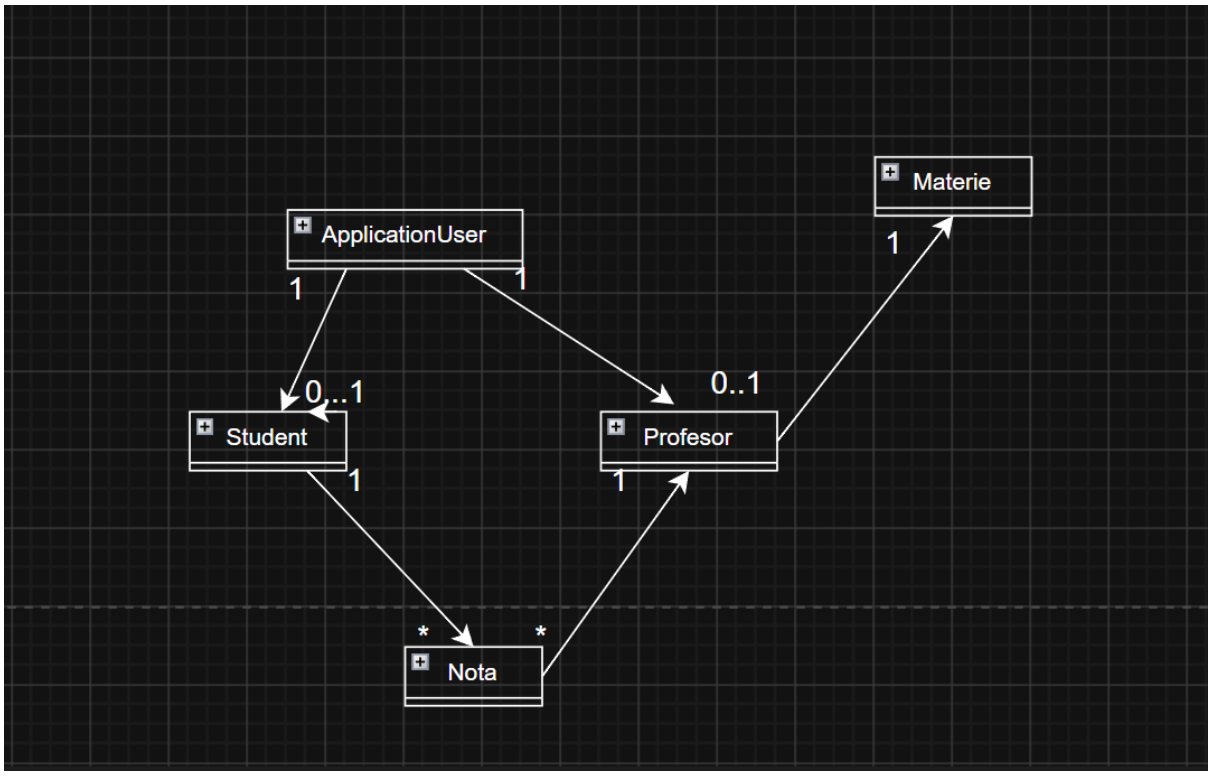
JavaScript → is a programming language and core technology of the Web, alongside HTML and CSS. The overwhelming majority of websites uses JavaScript for client-side webpage behaviour.

Microsoft SQL Server → is a proprietary relational database management system developed by Microsoft using Structured Query Language (SQL).

Deliverable 2

Domain Model

The component diagram highlights the main software components of the system. The user interface is implemented through ASP.NET Core MVC, using Views, Controllers, and ViewModels. Administrative functionalities and business operations are exposed through a separate backend, implemented as an ASP.NET Core Web API. The application logic is encapsulated in service classes, and database access is performed through Entity Framework Core and ApplicationDbContext. Authentication and authorization are managed through ASP.NET Core Identity.



Architectural Design

Conceptual Architecture

The system uses a layered architecture. The user interface is implemented in ASP.NET Core MVC, while the administrative functionalities and business operations are exposed through a separate backend of ASP.NET Core Web API type. Data persistence is achieved through Entity Framework Core and SQL Server. The architecture clearly separates responsibilities between presentation, logic, and data access.

Architectural Style : Layered Architecture and Client-Server Architecture

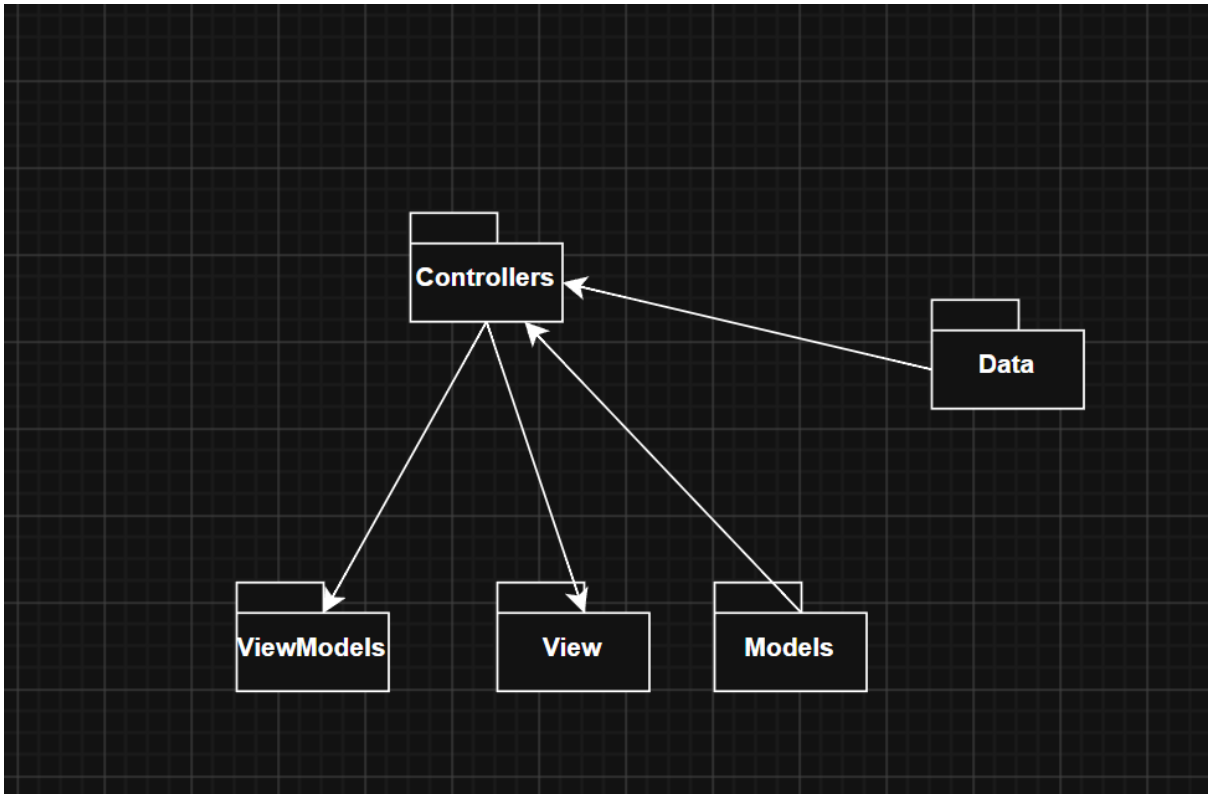
Patterns : MVC pattern, Service Layer Pattern, Repository-like behavior through DbContext/EF Core

I chose this type of architecture for reasons such as: separation of responsibilities, easier maintenance, scalability, the possibility of having separate backend and frontend.

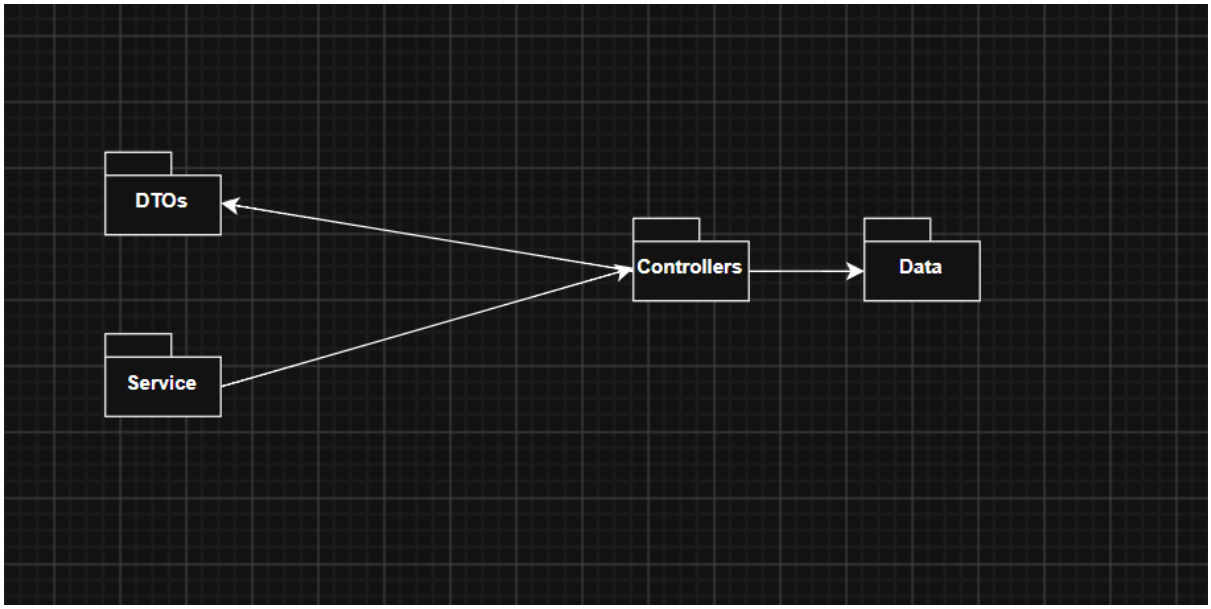
Package Design

In the Package Diagram we need to highlight the relationships between the packages within the program, both from MVC and from the API.

MVC



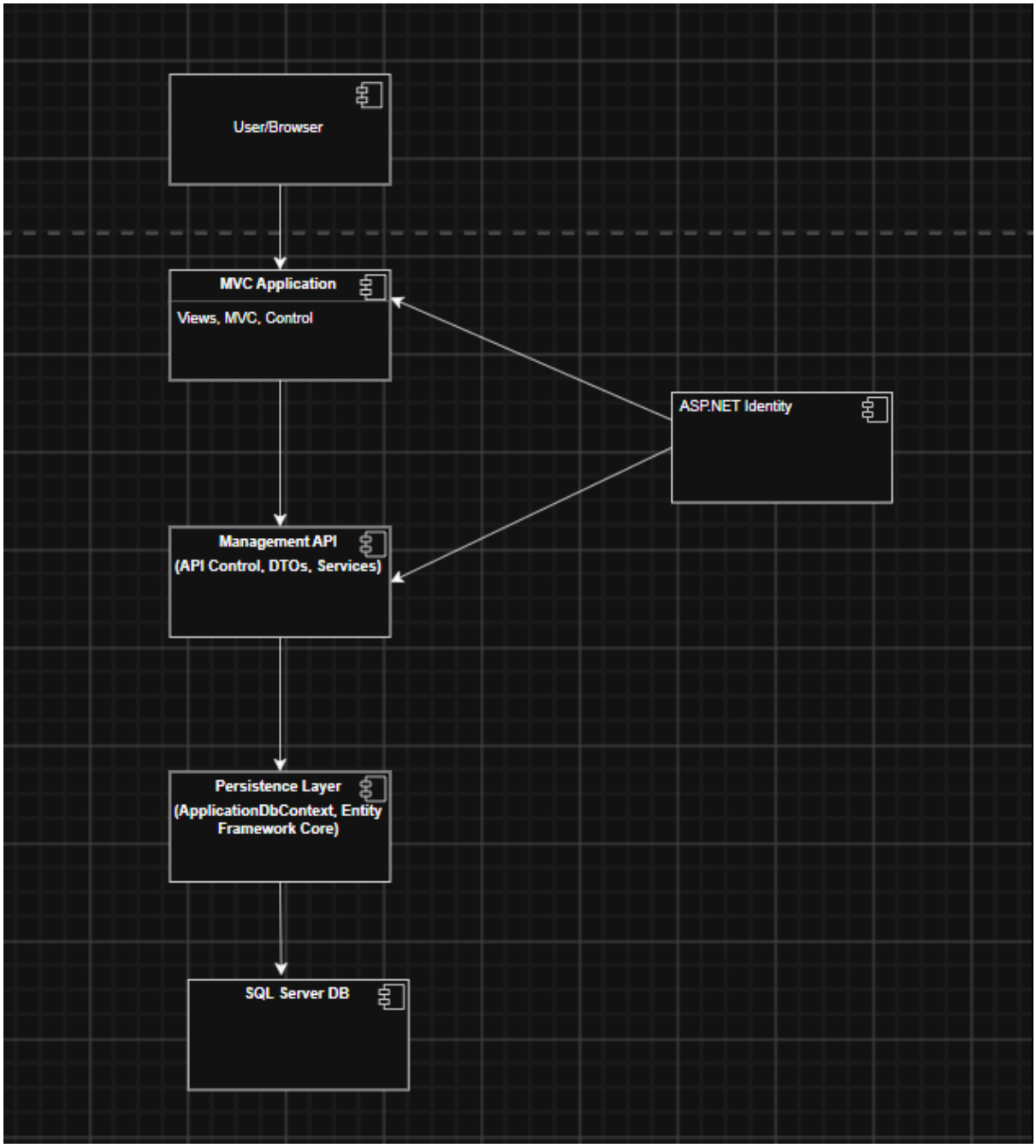
API



Component and Deployment Diagram

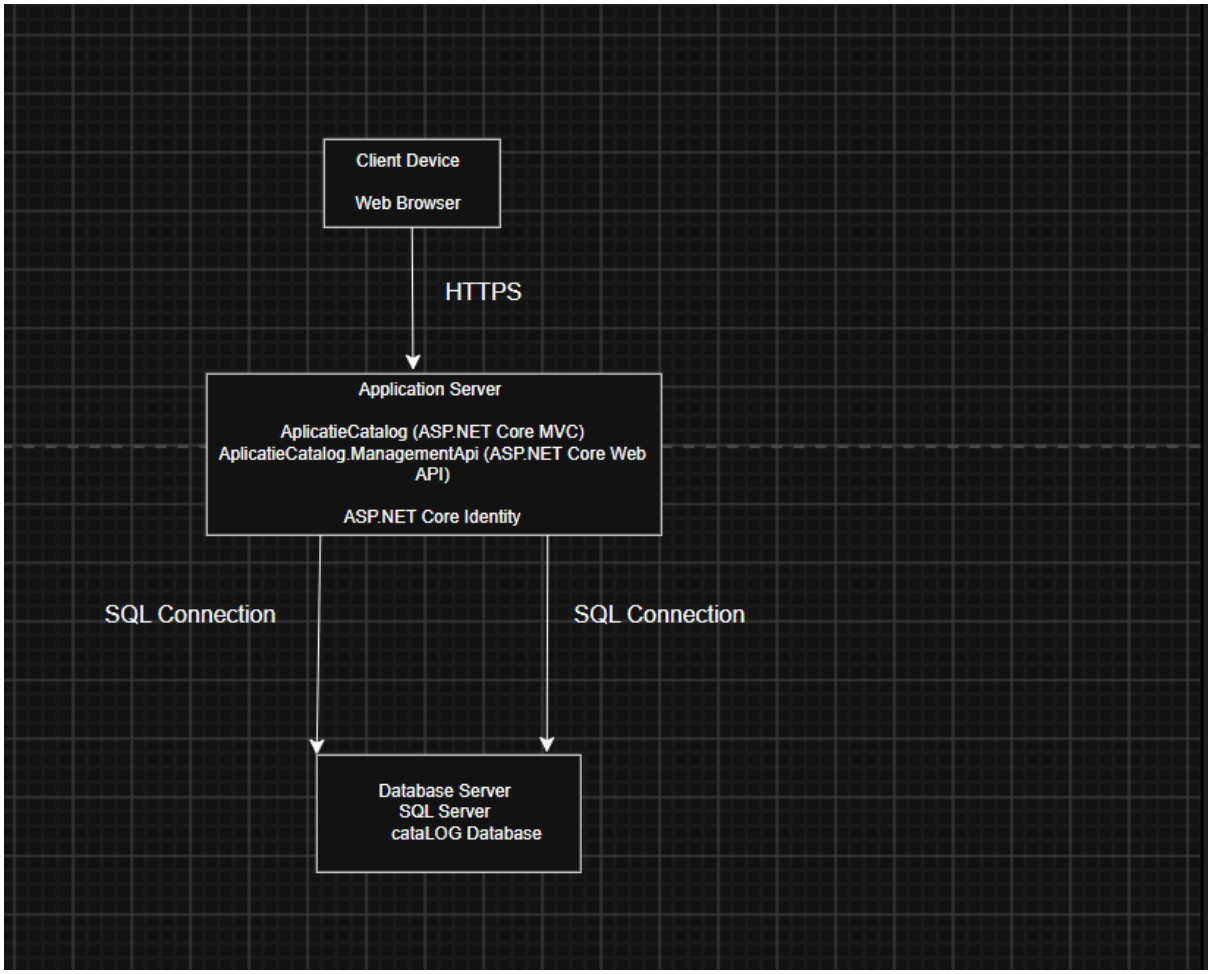
Component Diagram

The system is composed of two main software subsystems: the ASP.NET Core MVC application, responsible for the user interface, and the ASP.NET Core Web API, responsible for business operations such as user management, grade management, and subject administration. The MVC subsystem contains views, controllers, and view models, while the API subsystem contains controllers, DTOs, and service classes. Both subsystems interact with the same persistence layer through Entity Framework Core and the ApplicationDbContext. Authentication and authorization are handled using ASP.NET Core Identity.



Deployment Diagram

The deployment architecture contains three main nodes: the client browser, the application server, and the database server. The client accesses the MVC application through HTTPS. The MVC application communicates with the management API over HTTP/HTTPS. Both applications access the SQL Server database, which stores the persistent data of the system, including users, students, teachers, grades, and subjects.



Deliverable 3

Design Model

Dynamic Behavior

[Create the interaction diagrams (2 sequence) for 2 relevant scenarios]

Class Diagram

[Create the UML class diagram; apply GoF patterns and motivate your choice]

Data Model

[Create the data model for the system.]

System Testing

[Describe the testing methides and some test cases.]

Future Improvements

[Present some features that apply to the application scope.]

Conclusion

Bibliography